



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U04_A04 Casos prácticos, Ponte a prueba, etc – Prof.
Emilio Ramírez**

Autor: Segura Abad, Mario

Fecha: 1 de Febrero, 2025

INDICE

INTRODUCCIÓN	3
ENUNCIADO.....	4
PROBAD TODO EL CÓDIGO DEL APARTADO 4	4
PROBAD LOS SCRIPTS DEL APARTADO 4	4
CASO PRÁCTICO 4	4
<i>Trabajo con una API de servidor</i>	4
PONTE A PRUEBA 4	4
<i>Descarga de datos con tiempo máximo de espera</i>	4
CLAVES DE RESOLUCIÓN:	5
• <i>En primer lugar, intenta mostrar por la consola los datos recibidos.</i>	5
• <i>Observa qué formato de datos devolverá la petición.</i>	5
• <i>Una vez que hayas podido mostrar los datos en la consola, trata de formatear la salida, de acuerdo con las instrucciones.</i>	5
• <i>Cuando esté todo funcionando, añade una señal para poder la petición.</i>	5
• <i>Por último, añade el código para que la señal de abortar se active cuando pasen 5 segundos. Para ello, puedes utilizar un temporizador.</i>	5
DESARROLLO DE LA PRÁCTICA	6
DESCRIPCIÓN	6
• <i>Consultar la API pública de GitHub para obtener datos de un usuario (Caso práctico 4).</i>	6
• <i>Realizar una petición GET a un archivo JSON local con un tiempo máximo de espera y cancelación automática (Ponte a prueba 4).</i>	6
CONTENIDO DEL ARCHIVO	6
<i>Código del Apartado 4 de teoría</i>	6
<i>Caso Práctico 4 – Trabajo con una API de servidor (GitHub)</i>	9
<i>Ponte a Prueba 4 – Descarga de datos con tiempo máximo de espera</i>	10
CÓMO VISUALIZARLO	11
Requisitos:	11
Pasos:	11
OBJETIVOS DE LA PRÁCTICA	11
• <i>Probar y comprender el código teórico del apartado 4.</i>	11
• <i>Consumir APIs externas con fetch.</i>	11
• <i>Controlar tiempos de espera y cancelar peticiones con AbortController.</i>	11
• <i>Formatear datos JSON recibidos desde un servidor.</i>	11
• <i>Documentar correctamente, como siempre, cada ejercicio con nombre, fecha y capturas de pantalla.</i>	11
• <i>Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.</i>	11
CONCLUSIÓN	12

INTRODUCCIÓN

En el ejercicio “Probad todo el código del Apartado 4” deberemos incluir en varios fuentes (yo siempre recomiendo en varios) absolutamente todo el código que aparece en la teoría del apartado 4 de Programación asíncrona, así como su ejecución y la prueba de autoría.

En el ejercicio “Probad los Scripts del Apartado 4” hay que probar y ejecutar los scripts de dicho apartado y explicar cómo funciona. Incluiremos la autoría.

En el “Caso práctico 4” crearemos una página web que utilizará un formulario mostrando el login y avatar de usuario. Se utilizará la API de GitHub con su URL correspondiente.

En el “Ponte a prueba 4” crearemos un programa que realice una petición GET a través de un servidor web a un JSON y almacenará una lista de usuarios. Dispondremos del archivo DWECE_U04_A04_PP_E.json.

ENUNCIADO

Probad todo el código del Apartado 4

Incluid en uno o varios fuentes **todo** el código que aparece en la teoría de este apartado, así como su ejecución y prueba de autoría.

Probad los Scripts del Apartado 4

Ejecutad los scripts adjuntos y explicad su funcionamiento. Recordad incluir pruebas de autoría.

Caso práctico 4

Trabajo con una API de servidor

Crea una página web que utilice un formulario para mostrar el *login* y la imagen de avatar de un usuario de GitHub. El formulario tendrá un campo de texto para introducir el nombre de usuario y un botón de envío. Se utilizará la API de GitHub. La URL que se debe utilizar es <https://api.github.com/users/{USER}>, siendo **{USER}** el nombre de usuario. Por ejemplo, los datos del usuario **vuejs** están disponibles en <https://api.github.com/users/vuejs> (petición **GET**).

Los datos devueltos por la API están en formato JSON. Del objeto devuelto se utilizarán las propiedades **login** y **avatar_url**. Se debe indicar si el usuario no existe.

Tened cuidado al realizar las pruebas: la API de GitHub solo permite 60 peticiones a la hora sin autenticar.

Ponte a prueba 4

Descarga de datos con tiempo máximo de espera

Crea un programa que realice una petición **GET** a través de un servidor web a un archivo JSON que almacena un listado de usuarios. Para ello, crea una carpeta que almacena el archivo HTML que utilizarás para ejecutar el programa y el archivo JSON indicado y utiliza la extensión Live Server del Visual Studio Code para ejecutar un servidor sobre dicha carpeta.

Si la petición tarda más de 5 segundos, se deberá cancelar y deberá mostrarse un mensaje en la consola; si la petición tiene éxito, se deberá mostrar en la consola el listado de usuarios de la siguiente manera:

Usuario: Pérez, Laura.

Usuario: Martínez, Ana.

Usuario: González, Juan.

Adjunto dispones del **archivo DWECS_U04_A04_PP_E.json**, con el archivo JSON que almacena el listado de usuarios.

Claves de resolución:

- En primer lugar, intenta mostrar por la consola los datos recibidos.
- Observa qué formato de datos devolverá la petición.
- Una vez que hayas podido mostrar los datos en la consola, trata de formatear la salida, de acuerdo con las instrucciones.
- Cuando esté todo funcionando, añade una señal para poder la petición.
- Por último, añade el código para que la señal de abortar se active cuando pasen 5 segundos. Para ello, puedes utilizar un temporizador.

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 4 Apartado 4

Descripción

Este apartado se centra en el uso de **fetch**, APIs externas y control de peticiones HTTP en JavaScript.

El objetivo principal ha sido:

- Consultar la API pública de GitHub para obtener datos de un usuario (Caso práctico 4).
- Realizar una petición GET a un archivo JSON local con un tiempo máximo de espera y cancelación automática (Ponte a prueba 4).

Contenido del archivo

Código del Apartado 4 de teoría

Se ha probado todo el código relacionado con:

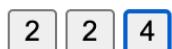
- Uso de `fetch()` para realizar peticiones HTTP.
- Conversión de respuestas a JSON mediante `.json()`.
- Manejo de errores con `try/catch` y comprobación de códigos de estado.
- Uso de `AbortController` para cancelar peticiones.
- Control de tiempo máximo de espera mediante `setTimeout`.

Cada archivo incluye comentarios con el nombre del autor y la fecha de realización.

Se han capturado los resultados por consola y pantalla como prueba de ejecución.

Ejemplo de ejecución:

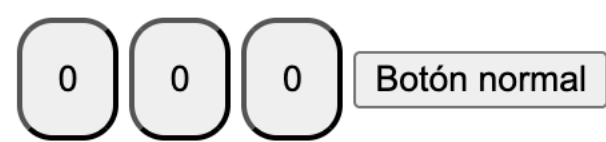
Salida por consola – DWEK_U04_A04_01.html



Autor: **Mario Segura Abad**

Fecha de realización: 20/01/2026

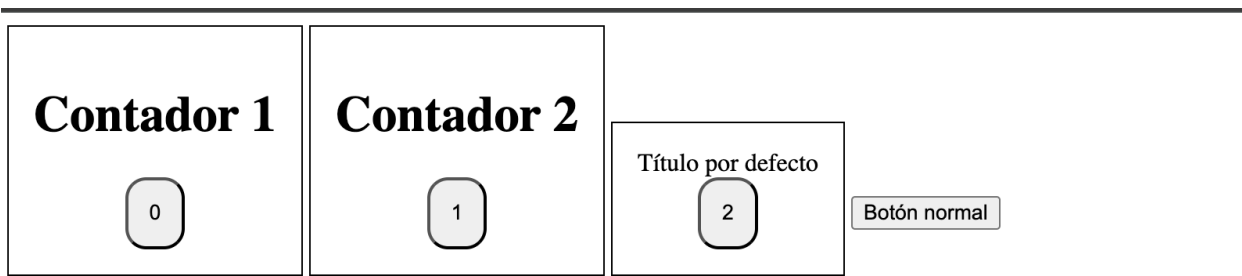
Salida por consola – DWEC_U04_A04_02.html



Autor: **Mario Segura Abad**

Fecha de realización: 20/01/2026

Salida por consola – DWEC_U04_A04_03.html



Autor: **Mario Segura Abad**

Fecha de realización: 20/01/2026

Salida por consola – DWEC_A04_U04_TEO_1.js

```
// Prueba de autoría
console.log("Autor: Mario Segura Abad");
console.log("Fecha de realización: 01/02/2026");
< undefined
```

Salida por consola – DWEC_U04_A04_TEO_2.js

```
programaPrincipal();  
Autor: Mario Segura Abad  
Fecha de realización: 19/01/2026  
< ▾ Promise {<pending>} ⓘ  
  ▶ [[Prototype]]: Promise  
  ▶ [[PromiseState]]: "rejected"  
  ▶ [[PromiseResult]]: TypeError: Failed to fetch at programaPrincipal (<anonymous>:8:27) at <anonymous>:30:1
```

Salida por consola – DWEC_U04_A04_TEO_3.js

```
Autor: Mario Segura Abad  
Fecha de realización: 19/01/2026  
< ▾ Promise {<pending>} ⓘ  
  ▶ [[Prototype]]: Promise  
  ▶ [[PromiseState]]: "pending"  
  ▶ [[PromiseResult]]: undefined
```

Salida por consola – DWEC_U04_A04_TEO_4.js

```
Autor: Mario Segura Abad  
Fecha de realización: 20/01/2026  
< ▾ Promise {<pending>} ⓘ  
  ▶ [[Prototype]]: Promise  
  ▶ [[PromiseState]]: "pending"  
  ▶ [[PromiseResult]]: undefined
```

Salida por consola – DWEC_U04_A04_TEO_5.js

```
Autor: Mario Segura Abad  
Fecha de realización: 20/01/2026  
< ▾ Promise {<pending>} ⓘ  
  ▶ [[Prototype]]: Promise  
  ▶ [[PromiseState]]: "pending"  
  ▶ [[PromiseResult]]: undefined
```


Salida por consola – DWEC_U04_A04_TEO_6.js

```
// Petición cancelada
Autor: Mario Segura Abad
Fecha de realización: 20/01/2026
Petición cancelada
< undefined
```

Caso Práctico 4 – Trabajo con una API de servidor (GitHub)

Objetivo:

Crear una página web con un formulario que permita introducir un nombre de usuario de GitHub y mostrar:

- Su **login**.
- Su **avatar** (avatar_url).

Si el usuario no existe, se debe mostrar un mensaje de error.

Claves de resolución:

- Utilizar la URL: <https://api.github.com/users{USER}>
- Capturar el envío del formulario y evitar la recarga.
- Realizar la petición con `fetch()`.
- Convertir la respuesta a JSON:

```
const datos = await respuesta.json();
```

- Comprobar si el usuario existe (`respuesta.status === 404`).
- Mostrar login e imagen en la página.
- Tener en cuenta el límite de **60 peticiones/hora** sin autenticar.

Este ejercicio ha reforzado el uso de **APIs REST**, manejo de respuestas JSON y control de errores en peticiones HTTP.

Ejemplo de ejecución:

Salida por consola – CasoPractico4.html

Buscar usuario de GitHub

Autor: **Mario Segura Abad**

Fecha de realización: 20/01/2026

user is not defined

Ponte a Prueba 4 – Descarga de datos con tiempo máximo de espera

Objetivo:

Realizar una petición GET a un archivo JSON local que contiene un listado de usuarios, con un tiempo máximo de espera de **5 segundos**.

Claves de resolución:

- Crear una carpeta con:
 - El HTML.
 - El script JS.
 - El archivo JSON (DWEU04_A04_PP_E.json).
- Ejecutar la carpeta con **Live Server** (Visual Studio Code).
- Realizar la petición con `fetch()`.
- Mostrar los datos en consola con el formato:
 - Usuario: Pérez, Laura.
 - Usuario: Martínez, Ana.
- Añadir un **AbortController**:

```
const controller = new AbortController();
const signal = controller.signal();
```

- Crear un temporizador de 5 segundos:

```
setTimeout(() => controller.abort(), 5000);
```
- Si la petición se cancela → mostrar mensaje en consola.
- Si tiene éxito → formatear y mostrar los resultados.

Este ejercicio ha reforzado el uso de **fetch avanzado**, abortos de petición y control de tiempos de espera.

Ejemplo de ejecución:

Salida por consola – PonteAPrueba4.html

Descarga de usuarios

Autor: **Mario Segura Abad**

Fecha de realización: 22/01/2026

```
Usuario: Pérez,Laura.  
Usuario: Martínez,Ana.  
Usuario: González,Juan.  
> |
```

Cómo visualizarlo

Requisitos:

- Editor de código: en mi caso, Visual Studio Code.
- Navegador web moderno: en mi caso, Google Chrome.

Pasos:

Caso práctico 4 (GitHub API):

1. Guardar .html y .js en la raíz del proyecto, **muy importante este paso**.
2. Abrir el proyecto con nuestro editor de código.
3. Ejecutar con Live Server (opcional).
4. Introducir un usuario de GitHub en el formulario.
5. Ver login y avatar en la pantalla o mensaje de error si no existe.
6. Revisar la consola (F12 → "Consola") para ver la prueba de autoría.

Ponte a prueba 4 (JSON con tiempo máximo)

1. Crear carpeta con HTML + JS + JSON.
2. Ejecutar la carpeta con Live Server.
3. Observar en consola:
 - a. Listado formateado si la petición es rápida.
 - b. Mensaje de cancelación si supera los 5 segundos.
4. Verificar mensajes de autoría.

Objetivos de la práctica

- Probar y comprender el código teórico del apartado 4.
- Consumir APIs externas con fetch.
- Controlar tiempos de espera y cancelar peticiones con AbortController.
- Formatear datos JSON recibidos desde un servidor.
- Documentar correctamente, como siempre, cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica ha permitido consolidar el uso de **fetch**, APIs REST y control avanzado de peticiones HTTP.

Se ha comprobado, asimismo, cómo consumir datos externos, cómo manejar errores y cómo cancelar peticiones cuando superan un tiempo límite.

La práctica nos ha sido muy útil para entender la comunicación cliente-servidor en aplicaciones web modernas.